

RAPPORT DE SAE – FIND MY WORD

Nom des membres de l'équipe :

Kande CISSOKO

Lucas RENTE-PINTO

Groupe: ceres

Sommaire:

1. Présentation du projet
2. Lancement du programme
3. Analyse et conception
4. Diagramme UML
5. Description des classes principales
6. Principes POO utilisés
7. Validation et tests
8. Organisation du travail
9. Difficultés rencontrées
10. Bilan du projet

1. Présentation du projet

Le projet consiste à développer une version simplifiée du jeu FindMyWord, inspiré du célèbre jeu Wordle. Le principe du jeu est simple : le joueur doit retrouver un mot secret composé de cinq lettres en un maximum de six tentatives.

Après chaque proposition du joueur, le programme analyse les lettres du mot saisi et indique leur état :

- OK : la lettre est correcte et bien placée ;
- PRESENT : la lettre existe dans le mot secret mais n'est pas à la bonne position ;
- ABSENT : la lettre n'existe pas dans le mot secret.

L'objectif principal du projet était de mettre en pratique les concepts de programmation orientée objet en Java, tout en réalisant une application fonctionnelle et structurée.

2. Lancement du programme

Le programme s'exécute directement depuis un terminal à l'aide des commandes Java classiques.

Exemple d'exécution :

java Main

Aucune utilisation d'un IDE n'est nécessaire pour lancer l'application. Le projet a été conçu de manière à fonctionner uniquement avec les outils standards fournis par Java.

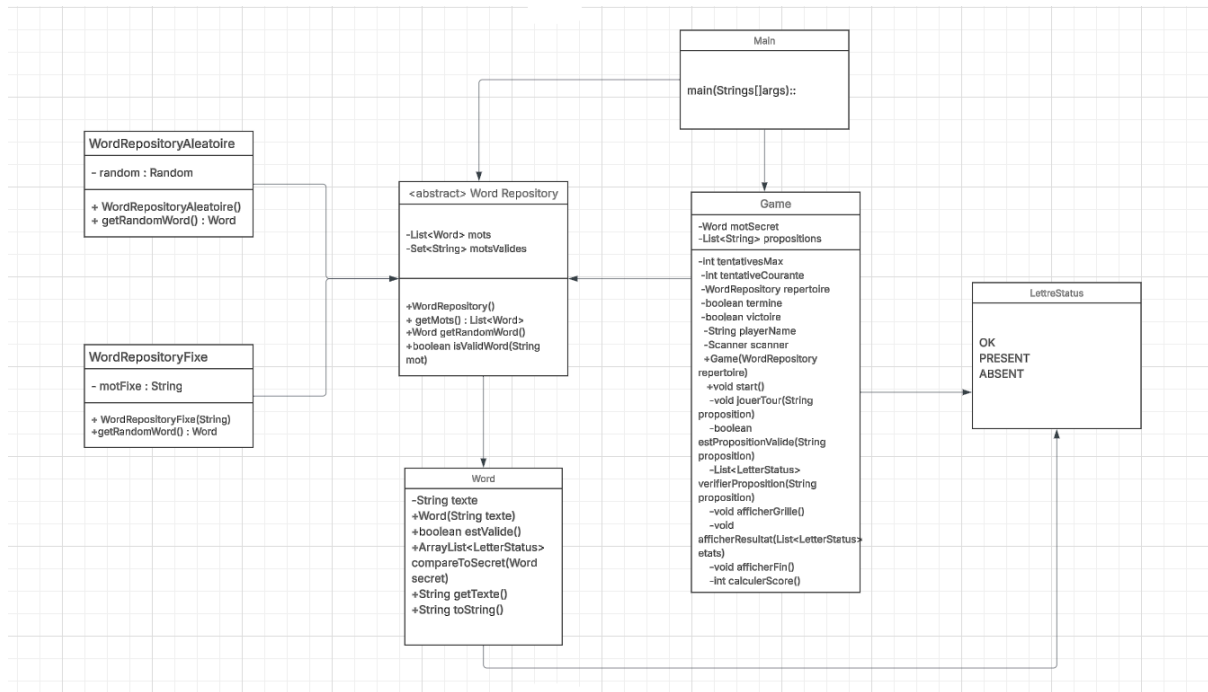
3. Analyse et conception

Une approche orientée objet a été utilisée afin de concevoir une architecture claire et organisée. Le projet a été découpé en plusieurs classes possédant chacune une responsabilité précise.

Cette séparation permet de rendre le code plus lisible, plus simple à maintenir et plus facile à faire évoluer par la suite. Chaque classe intervient dans une partie spécifique du fonctionnement du jeu, ce qui limite les dépendances inutiles entre les différentes parties du programme.

La conception du projet a également nécessité une réflexion préalable sur l'organisation générale des données et sur les interactions entre les objets.

4. Diagrammes UML



Classes principales

- Main : point d'entrée du programme permettant de démarrer l'application ;
- Game : gestion complète de la partie et des interactions avec le joueur ;
- Word : représentation d'un mot et validation des règles associées ;
- WordRepository : récupération des mots utilisés par le jeu depuis la source de données.

5. Description des classes principales :

Game

La classe "Game" gère l'ensemble du déroulement d'une partie. Elle contrôle la boucle de jeu, le nombre de tentatives restantes, les conditions de victoire ou de défaite ainsi que les échanges avec l'utilisateur.

Word

La classe "Word" représente les mots manipulés dans le programme. Elle permet notamment de vérifier la validité des saisies et d'appliquer les règles métier liées au jeu.

WordRepository

La classe "WordRepository" est chargée de fournir les mots du dictionnaire utilisés pendant les parties. Elle centralise la récupération des données afin de simplifier le reste du programme.

Main

La classe "Main" constitue le point de départ de l'application. Elle initialise les objets nécessaires puis lance une nouvelle partie.

6. Principes POO utilisés

Plusieurs principes fondamentaux de la programmation orientée objet ont été utilisés lors du développement du projet.

Encapsulation

Les attributs des classes sont privés afin de protéger les données internes des objets. L'accès à ces informations se fait uniquement par des méthodes adaptées.

Héritage

L'héritage est utilisé lorsque cela est pertinent afin de spécialiser certains comportements et d'éviter la duplication de code.

Polymorphisme

Le polymorphisme permet de remplacer certaines implémentations sans modifier le reste du programme, ce qui améliore la flexibilité du code.

Séparation des responsabilités

Chaque classe possède une mission précise et indépendante. Cette organisation rend le projet plus clair et facilite la maintenance du programme.

exemple avec `WordRepository` , `WordRepositoryAleatoire` et `WordRepositoryFixe`

D'abord `WordRepository`:

Cette classe contient tout ce qui est commun aux différents modes de récupération des mots.

Elle charge les mots depuis le fichier JSON, stocke les mots dans une liste, elle vérifie si un mot est valide et définit la méthode `getRandomWord()` sans l'implémenter. En étant une classe abstraite, elle sert de base pour les deux autres classes qui héritent d'elle .

Ensuite `WordRepositoryAleatoire` :

Elle choisit un mot au hasard parmi la liste de mot disponible, à chaque nouvelle partie, un mot différent peut être sélectionné. Cette classe correspond au fonctionnement de base du jeu.

Puis `WordRepositoryFixe`:

Cette classe retourne toujours le même mot, elle nous permet de vérifier que le jeu marche, que toutes les fonctionnalités soient opérationnelles et faire des tests de débogages. Par exemple, si on connaît déjà le mot à trouver, on peut vérifier plus facilement que les lettres sont correctement détectées.

Le polymorphisme est présent grâce à la classe abstraite `WordRepository`. En effet, une variable de type `WordRepository` peut référencer aussi bien un objet `WordRepositoryAleatoire` qu'un objet `WordRepositoryFixe`.

Lorsque la méthode `getRandomWord()` est appelée sur cette variable, Java exécute automatiquement l'implémentation correspondant au type réel de l'objet. Ainsi, selon que l'objet est aléatoire ou fixe, la méthode retournera soit un mot choisi aléatoirement, soit un mot prédéfini.

Cette capacité à utiliser une même référence pour manipuler plusieurs objets ayant des comportements différents est concrètement du polymorphisme.

exemple concret :

```
WordRepository repo1 = new WordRepositoryAleatoire();  
WordRepository repo2 = new WordRepositoryFixe("agent");
```

7. Validation et tests

Plusieurs tests ont été réalisés afin de vérifier le bon fonctionnement de l'application et de limiter les erreurs.

Les tests effectués couvrent notamment :

- la validation de la taille des mots ;
- la vérification des caractères alphabétiques ;
- la gestion des lettres répétées ;
- l'analyse correcte des états OK, PRESENT et ABSENT ;
- la gestion des situations de victoire et de défaite ;
- la non-comptabilisation des saisies invalides dans le nombre de tentatives.

Ces tests ont permis de corriger progressivement les erreurs rencontrées pendant le développement et d'améliorer la fiabilité générale du programme.

8. Organisation du travail

Le projet a été réalisé en binôme. Une répartition des tâches a été mise en place afin d'organiser efficacement le développement du projet.

kandé: UML ,Game, WordRepository , WordRepositoryAleatoire et WordRepositoryFixe

Lucas: Game , Word, Wordset_bundle_etudiant ,WordRepository

Les différentes tâches comprenaient :

- la modélisation UML ;
- le développement des classes ;
- le développement des interactions utilisateur ;
- la réalisation des tests ;
- la rédaction du rapport.

Les décisions techniques importantes ont été discutées conjointement afin de conserver une cohérence globale dans le projet et dans l'organisation du code. Chacun de nous faisaient une méthode notamment dans les grande classe comme game et WordRepository afin de pouvoir avancer plus vite et aider l'autre lorsque une méthode était complexe .

9. Difficultés rencontrées

Plusieurs difficultés ont été rencontrées au cours du développement du projet.

Les principales difficultés concernaient :

- la modélisation initiale des classes ;
- la validation des mots saisis ;
- l'analyse des lettres et de leur position ;
- l'organisation générale du projet.

Ces problèmes ont été résolus progressivement grâce à de nombreux tests, à des corrections régulières et à une amélioration continue du code au fur et à mesure de l'avancement du projet.

10. Bilan du projet

Ce qui a été réalisé :

Les principales fonctionnalités demandées ont été développées :

- gestion complète d'une partie FindMyWord ;
- validation des entrées utilisateur ;
- analyse des propositions ;
- détection des conditions de victoire et de défaite.

Ce qui n'a pas été réalisé

Certaines fonctionnalités optionnelles n'ont pas été développées, notamment :

- l'historique des parties ;
- le chronométrage ;
- le mode multijoueur.

Ce qui fonctionne

L'ensemble des fonctionnalités principales demandées dans le sujet fonctionne correctement et permet de jouer à une partie complète du jeu.

Ce qui reste à améliorer

L'ergonomie générale du programme ainsi que certaines fonctionnalités plus avancées pourraient être améliorées avec davantage de temps de développement.